4 Arrays and Lists Homework

Sai Ram Vodnala

06-20-2024

General instructions.

There are four exercises below. Three exercises are worth 10 points each, and are labelled (Either R, Python or Julia). For these exercises, you can implement a solution in the language of your choice. I've included code chunk templates for all three languages, but you only need to fill in the chunks for one language.

One exercise is labelled (R, Python and Julia). This exercise is worth 20 points, and you must provide a solution using all three languages. This exercise also includes code chunk templates for all three languages.

Some of the exercises have questions relating to the results. Write your answers in the narrative part of the text, formatted as *italicized* text, **bold** text or

> quoted text.

Submit both `Rmd` and typeset results using file name formats as in previous exercises.

I've defined variables you may use for some of the exercises below. If you choose Python, you may need to convert some of these variables to `numpy.array`.

```
CaloriesPerRecipeMean <- c(2123.8, 2122.3, 2089.9, 2250.0, 2234.2, 2249.6, 3051.9)
ServingsPerRecipeMean <- c(12.9, 12.9, 13.0, 12.7, 12.4, 12.4, 12.7)
CaloriesPerServingMean <- c(268.1, 271.1, 280.9, 294.7, 285.6, 288.6, 384.4)
CaloriesPerServingSD <- c(124.8, 124.2, 116.2, 117.7, 118.3, 122.0, 168.3)
Year <- c(1936, 1946, 1951, 1963, 1975, 1997, 2006)
```

R

```
CaloriesPerRecipeMean = [2123.8, 2122.3, 2089.9, 2250.0, 2234.2, 2249.6, 3051.9]
ServingsPerRecipeMean = [12.9, 12.9, 13.0, 12.7, 12.4, 12.4, 12.7]
CaloriesPerServingMean = [268.1, 271.1, 280.9, 294.7, 285.6, 288.6, 384.4]
CaloriesPerServingSD = [124.8, 124.2, 116.2, 117.7, 118.3, 122.0, 168.3]
Year = [1936, 1946, 1951, 1963, 1975, 1997, 2006]
```

Python

```
CaloriesPerRecipeMean = [2123.8, 2122.3, 2089.9, 2250.0, 2234.2, 2249.6, 3051.9];
ServingsPerRecipeMean = [12.9, 12.9, 13.0, 12.7, 12.4, 12.4, 12.7];
CaloriesPerServingMean = [268.1, 271.1, 280.9, 294.7, 285.6, 288.6, 384.4];
CaloriesPerServingSD = [124.8, 124.2, 116.2, 117.7, 118.3, 122.0, 168.3];
Year = [1936, 1946, 1951, 1963, 1975, 1997, 2006];
```

Julia

Exercise 1 (Either R, Python or Julia)

Part a

Using vector operations or list comprehensions to check the values in Wansink, Table 1. The data are defined in the header chunk above. Divide each element in `CaloriesPerRecipeMean` by the corresponding element in `CaloriesPerServingMean`. How does the resulting vector (or list) compare to the values in `ServingsPerRecipeMean`?

```
ratio <- CaloriesPerRecipeMean / CaloriesPerServingMean
ratio
```

R

```
## [1] 7.921671 7.828477 7.440014 7.634883 7.822829 7.794872 7.939386
```

```
print(ratio == ServingsPerRecipeMean)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

Part b

Compute the mean of `CaloriesPerRecipeMean` and divide that value by the mean of `CaloriesPerServingMean`. How does this compare to the mean of the vector produced in Part a? How does this compare with the mean of `ServingsPerRecipeMean`? What does this suggest about the accuracy of Wansink, et al, Table 1?

```
mean_calories_per_recipe = mean(CaloriesPerRecipeMean)
mean_calories_per_serving = mean(CaloriesPerServingMean)
mean_ratio <- mean_calories_per_recipe / mean_calories_per_serving
print(mean_ratio)
```

R

```
## [1] 7.77549
```

```r
print(mean(ratio))
```

```
## [1] 7.768876
```

```r
print(mean(ServingsPerRecipeMean))
```

```
## [1] 12.71429
```

Exercise 2 (Either R, Python or Julia)

In this exercise, we will test your `BinomPMF` function with a range of inputs.

Do not print the vectors you create for this exercise in the final typeset submission We will check the results by examining the plots, and printing the vectors themselves will unnecessarily clutter your report. If you get stuck, use the built binomial functions to create your plots.

You should have defined functions named `BinomPMF` for each language - R, Python and Julia in the previous homework. Enter the function for the language you choose here.

```r
# function definition
BinomPMF <- function(k, n, pi) {
factorial(n) / (factorial(k) * factorial(n - k)) * pi^k * (1 - pi)^(n - k)
}
```

R

Part a.

Generate a sequence of values from $0, ..., 10$; let this be `X1`. Use your `BinomPMF` function to compute probabilities for each value in `X1`, using function vectorization, list comprehension or broadcasting as appropriate, and let the results be stored in a vector `Y1`. Let $n = 10$, and $\pi = 0.5$.

Show that `BinomPMF` produces a proper probability density by showing that `Y1` sums to 1.

Plot `Y1` versus `X1` (`X1` is the independent variable) as *blue* points.

Create a second vector `Y2`, similar to `Y1`, except let $\pi = 0.3$. Show that `Y2` sums to 1, and add a plot of `Y2` vs `X1` created in the previous step, but use *red* points. Show this plot.

```r
library(ggplot2)
```

```r
X1 <- 0:10
Y1 <- BinomPMF(X1, 10, 0.5)
sum_Y1 <- sum(Y1)
print(sum_Y1)
```

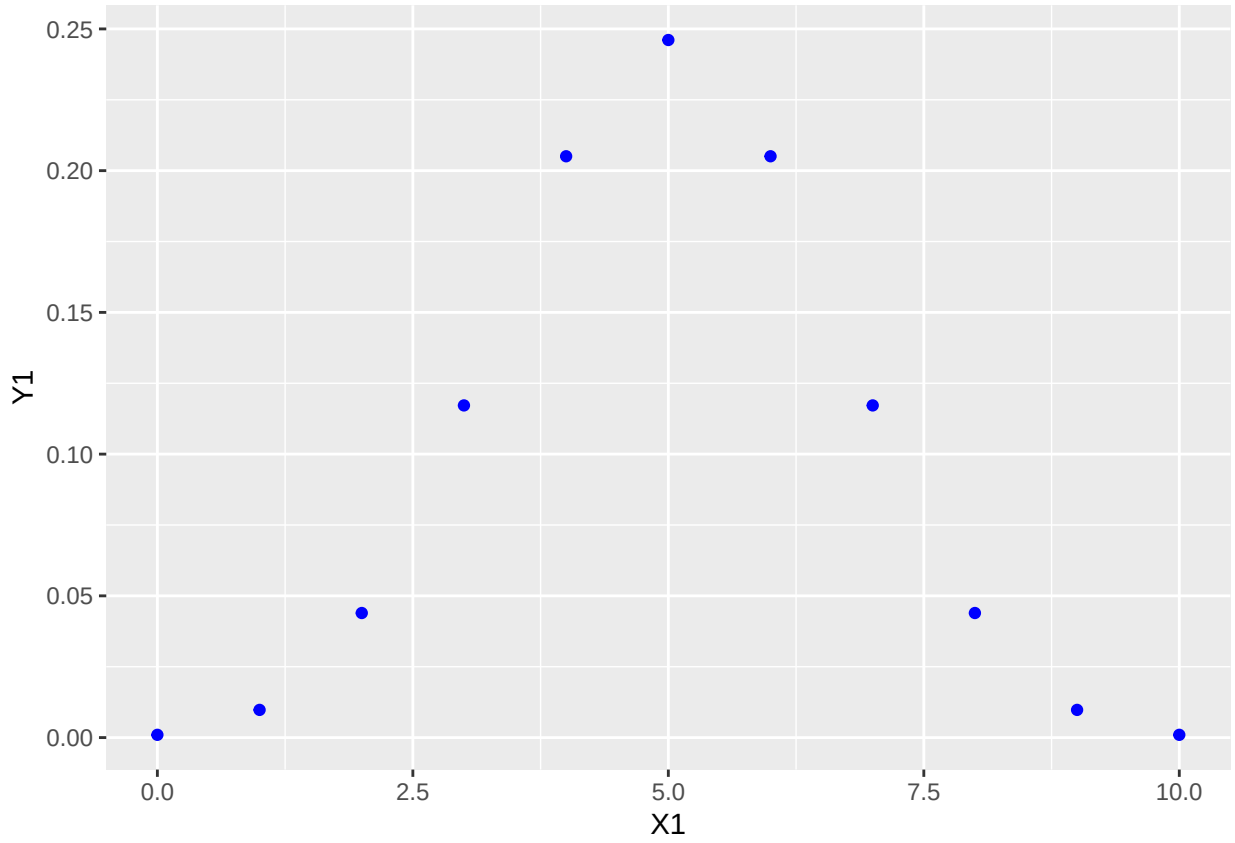
R

```
## [1] 1
```

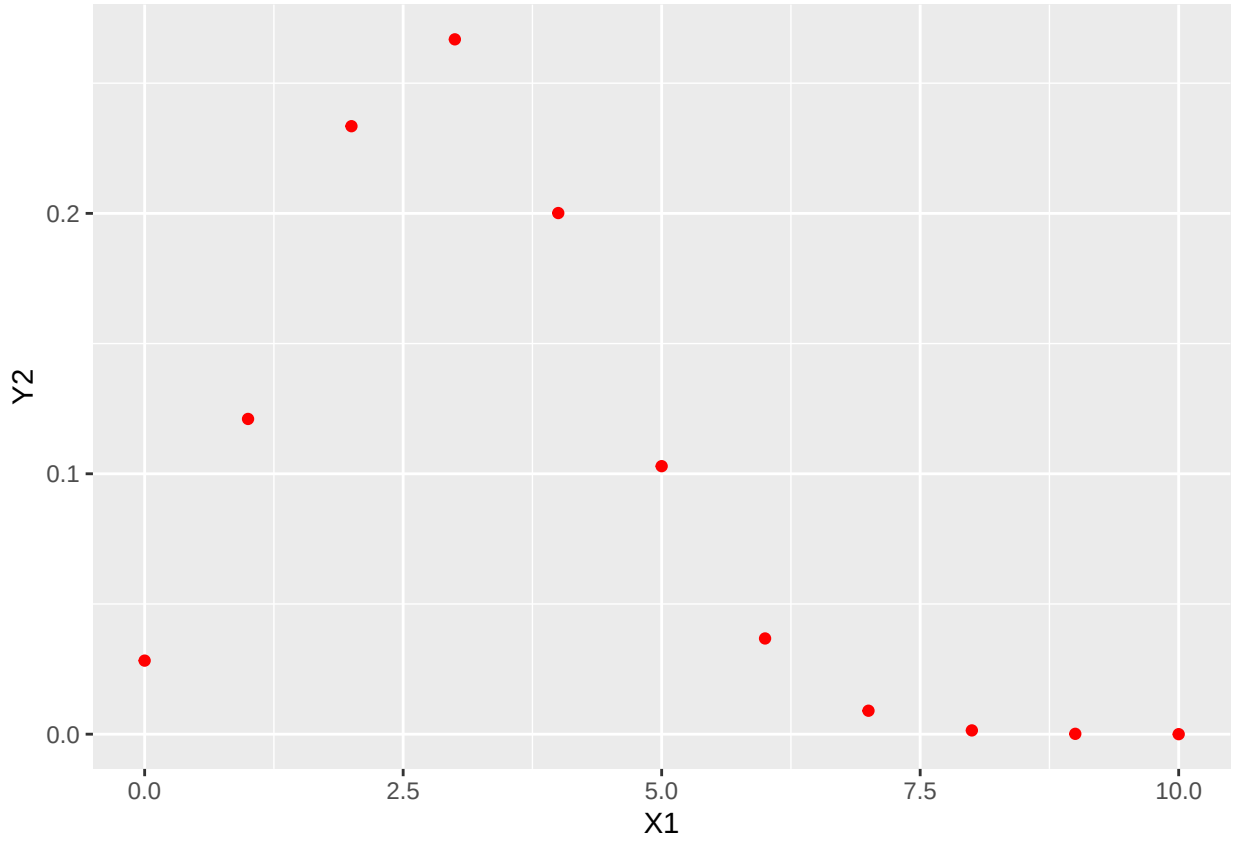
```
Y2 <- BinomPMF(X1, 10, 0.3)
sum_Y2 <- sum(Y2)
print(sum_Y2)
```

```
## [1] 1
```

```
df <- data.frame(X1, Y1)
ggplot(df, aes(x = X1, y = Y1)) + geom_point(color = "blue")
```




```
ggplot(df, aes(x = X1, y = Y2)) + geom_point(color = "red")
```



Part b.

When n is large, and π is near 0.5, the binomial distribution can be approximated using a normal distribution, with $\mu = n \times \pi$ and $\sigma = \sqrt{n \times \pi \times (1 - \pi)}$ To illustrate this, we'll create two more plots.

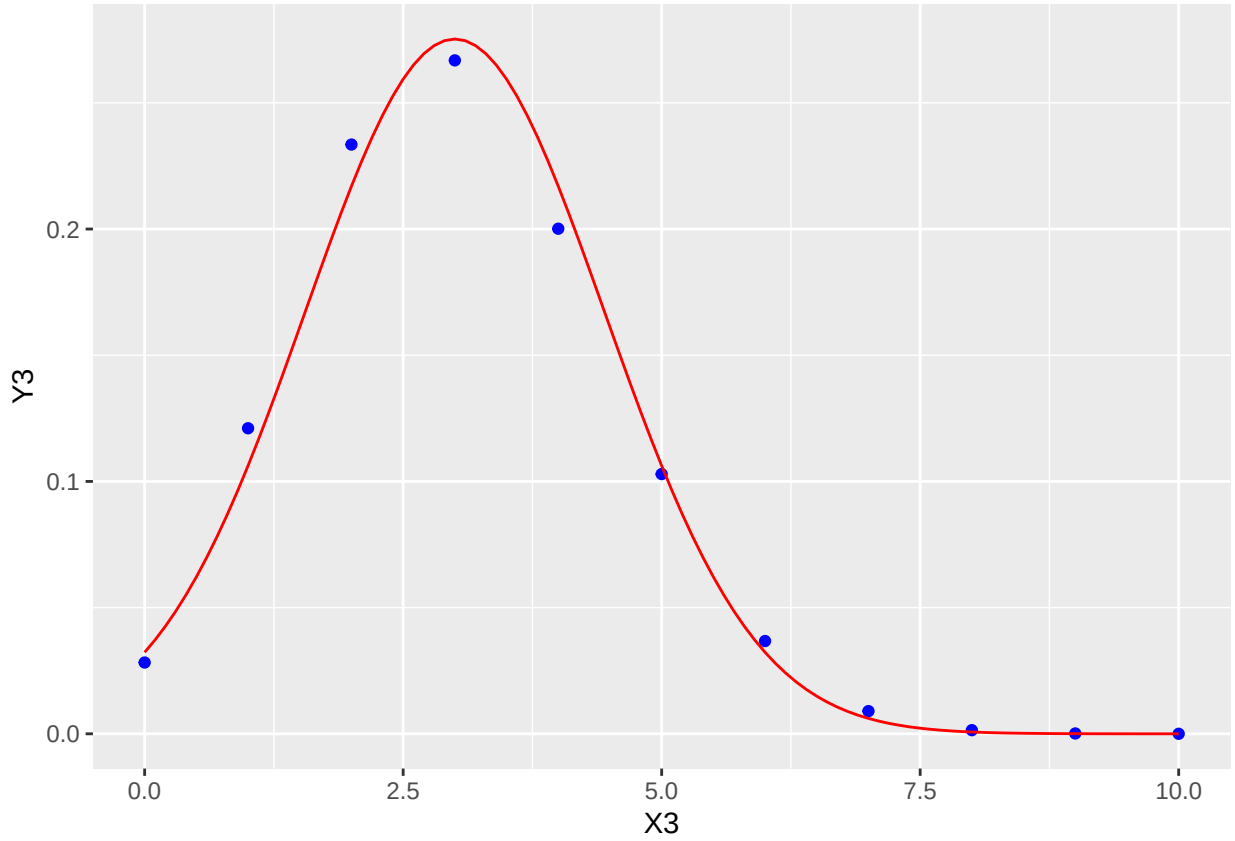
For the first plot, create a sequence `X3` from $0, ..., 10$, and use your `BinomPMF` function to create a vector `Y3` from the values in `X3` as in Part a. Let $n = 10$ and let $\pi = 0.3$. Create a sequence `X4` from $0, ..., 10$ incremented by 0.1. Compute a vector `Y4` by calling the normal pdf functions (`dnorm`, `scipy.stats.norm.pdf` and `pdf(Normal(...),...)`) using `X4` as the `x` argument. Let $\mu = n \times \pi$ and $\sigma = \sqrt{n \times \pi \times (1 - \pi)}$. It might be useful to define variables for n and π.

Plot `Y3` versus `X3` as *blue* points. Add a plot of `Y4` versus `X4` as *red* lines. Show this plot.

```
n <- 10
pi <- 0.3
X3 <- 0:10
Y3 <- BinomPMF(X3, n, pi)
X4 <- seq(0, 10, by = 0.1)

mu <- n * pi
sigma <- sqrt(n * pi * (1 - pi))
Y4 <- dnorm(X4, mean = mu, sd = sigma)
df1 <- data.frame(X3, Y3)
df2 <- data.frame(X4, Y4)
p <- ggplot() +
geom_point(data = df1, aes(x = X3, y = Y3), color = "blue") +
```

```
geom_line(data = df2, aes(x = X4, y = Y4), color = "red")
print(p)
```

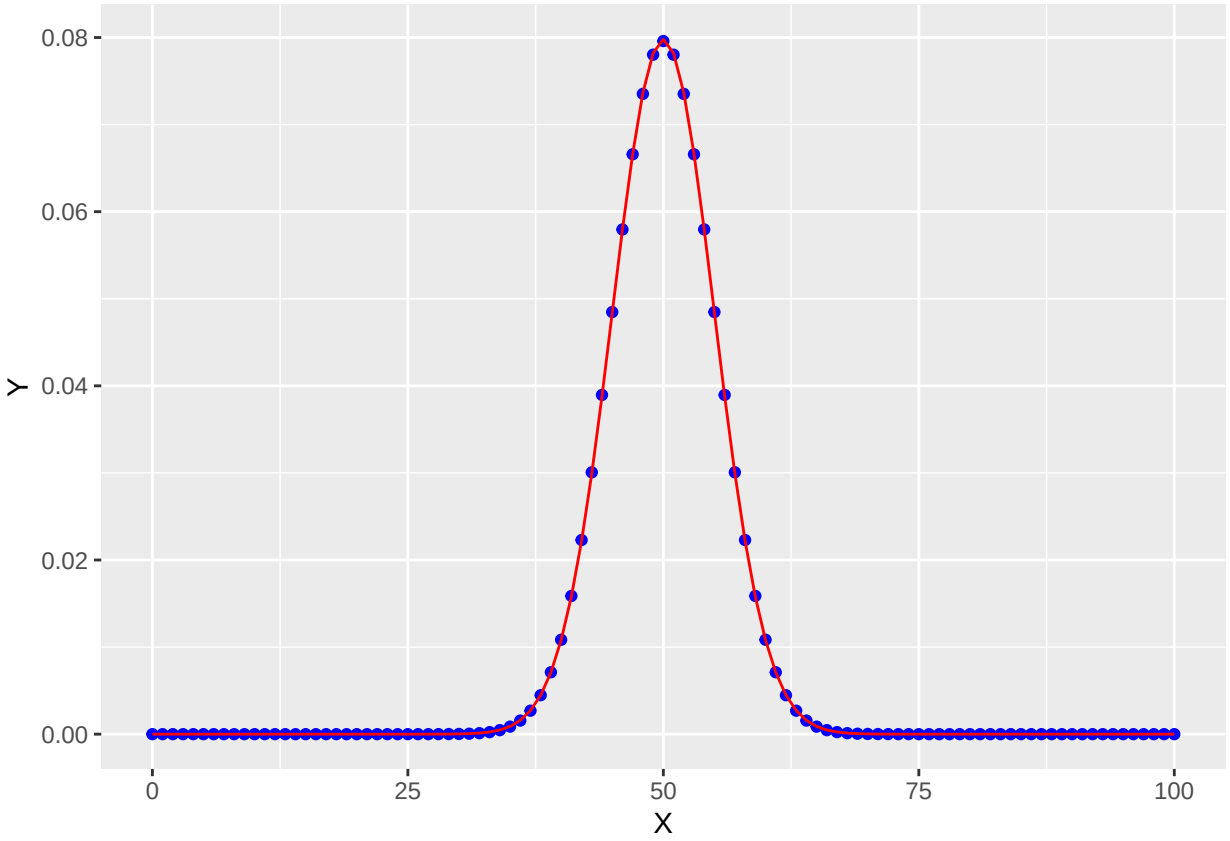


Part c.

Reproduce the plot in Part b, but this time, create a sequence from $0, ..., 100$ for the `X` vectors, and use $n = 100$ and $\pi = 0.5$. Show the plots here.

```
n <- 100
pi <- 0.5
X <- 0:100
Y <- BinomPMF(X, n, pi)

mu <- n * pi
sigma <- sqrt(n * pi * (1 - pi))
Y4 <- dnorm(X, mean = mu, sd = sigma)
df1 <- data.frame(X, Y)
df2 <- data.frame(X, Y4)
p <- ggplot() +
geom_point(data = df1, aes(x = X, y = Y), color = "blue") +
geom_line(data = df2, aes(x = X, y = Y4), color = "red")
print(p)
```



Which of the last two plots shows a better approximation of the binomial distribution? *Based on my observation, the second plot shows a better approximation of the binomial distribution. It is more symmetric and well-fitted, characteristics that are expected in a binomial distribution especially when the number of trials is large and the probability of success is near 0.5*

Exercise 3 (Either R, Python or Julia)

Given a vector of mean estimates $\mathbf{x} = x_1, x_2, \ldots, x_k$, a vector of standard deviations $\mathbf{s} = s_1, s_2, \ldots, s_k$ and a vector of sample sizes $\mathbf{n} = n_1, n_2, \ldots, n_k$, we can calculate a one-way analysis of variance by

$$MSB = \frac{n_1(x_1 - \bar{x})^2 + n_2(x_2 - \bar{x})^2 + \cdots + n_k(x_k - \bar{x})^2}{k-1} = \frac{\sum_i n_i(x_i - \bar{x})^2}{k-1}$$

and

$$MSW = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2 + \ldots (n_k - 1)s_k^2}{N-k} = \frac{\sum_i (n_i - 1)s_i^2}{N-k}$$

where $\bar{x} = \frac{\sum_i n_i * x_i}{N}$ and $N = \sum_i n_i$. The test statistic is $F = \frac{MSB}{MSW}$ which is distributed as $F_{k-1,N-k}$

Part a

Calculate MSW and MSB for Calories per Serving from Wansink Table 1, where $x_1 = $ the mean of Calories per Serving, 1936, and $s_1 = $ the standard deviation of Calories per Serving, 1936, etc. You can

use the variables `CaloriesPerServingMean` and `CaloriesPerServingSD` defined above. Let $n_1 = n_2 ... = n_k = 18$

Use array functions and arithmetic, or list comprehensions or broadcasting, for your calculations, you should not need explicit iteration (for loops outside list comprehensions). Do not hard code values for N and k, calculate these from the `CaloriesPerServingMean` or `CaloriesPerServingSD`. You may convert Python lists to `numpy` arrays if you choose Python.

Print both MSB and MSW.

```
n <- 18
k_mean <- length(CaloriesPerServingMean)
k_sd <- length(CaloriesPerServingSD)
x_mean <- CaloriesPerServingMean
sd <- (CaloriesPerServingSD)
N <- n*k_sd
Total_mean <- n*((x_mean-mean(x_mean))^2)
MSB <- sum(Total_mean)/(k_mean-1)
print(MSB)
```

R

```
## [1] 28815.96
```

```
total_msw <- (n-1)*(sd^2)
MSW <- sum(total_msw)/(N-k_sd)
print(MSW)
```

```
## [1] 16508.6
```

```
F <-MSB/MSW
print(F)
```

```
## [1] 1.745512
```

Part b

Calculate an F-ratio and a p for this F, using the F distribution with $k-1$ and $N-k$ degrees of freedom. You will need to look up the functions in R, Python or Julia that correspond to the F distribution (hint - consider how we computed p-values for Wald's and t-tests). Compare these values to the corresponding values reported in Wansink Table 1.

```
p_value <- pf(F, k_sd-1, N-k_sd, lower.tail = FALSE)
print(p_value)
```

R

```
## [1] 0.1163133
```

You can also check results by entering appropriate values in an online calculator like http://statpages.info/anova1sm.html .

Comment on why your calculated F and p values do not match the reported values in Wansink and Payne.

Exercise 4 (R, Python and Julia)

Suppose we wish to determine the relationship between per Calories per Serving and Year. We can determine this by solving a system of linear equations, of the form

$$\begin{aligned} 268.1 &= \beta_1 + \beta_2 \times 1936 \\ 271.1 &= \beta_1 + \beta_2 \times 1946 \\ \vdots &= \vdots \\ 384.4 &= \beta_1 + \beta_2 \times 2006 \end{aligned}$$

We write this in matrix notation as

$$\begin{pmatrix} 268.1 \\ 271.1 \\ \vdots \\ 384.4 \end{pmatrix} = \begin{pmatrix} 1 & 1936 \\ 1 & 1946 \\ \vdots & \vdots \\ 1 & 2006 \end{pmatrix} \begin{pmatrix} \beta_1 \\ \beta_2 \end{pmatrix}^t$$

We can also write this as

$$\mathbf{y} = \mathbf{X}\beta$$

and find a solution by computing $\widehat{\beta} = \mathbf{X}^{-1}\mathbf{y}$.

However, an exact solution for the inverse, $\mathbf{X}^{-1}$ require square matrices, so commonly we use the *normal* equations,

$$\mathbf{X}^t\mathbf{y} = \mathbf{X}^t\mathbf{X}\beta$$

(where $\mathbf{X}^t$ is the transpose of $\mathbf{X}$). We then find

$$\widehat{\beta} = \left(\mathbf{X}^t\mathbf{X}\right)^{-1}\mathbf{X}^t\mathbf{y}$$

Answer

Define appropriate `X` and `Y` matrices (I find `Y` can be copied from `CaloriesPerServingMean` in R and Julia, but needs to be constructed as a `numpy.array` in Python) in the chunks below.

Multiply the transpose of `X` by `X`, then use `solve` (R), `numpy.linalg.inv` (Python) or `inv` (Julia) to find the inverse $\left(\mathbf{X}^t\mathbf{X}\right)^{-1}$. Multiply this by the product of `X` transpose and `y` to find `HatBeta`.

Print your `HatBeta`.

```r
x <- matrix(Year,byrow = FALSE)
X <- matrix(c(rep(1,7),x), ncol = 2)
Y<- matrix(CaloriesPerServingMean, byrow = FALSE)
tX <- t(X)
matrix_m <- tX %*% X
HatBeta <- solve(matrix_m, tX %*% Y)
HatBeta
```

R

```
##              [,1]
## [1,] -1965.37261
## [2,]     1.14934
```

```python
import numpy as np
from scipy.linalg import inv
Y = np.array(CaloriesPerServingMean)
X = np.vstack((np.ones(len(Year)),Year)).T
X_transpose = X.T
XtX = X_transpose.dot(X)
XtX_inverse = inv(XtX)
XtY = X_transpose.dot(Y)
HatBeta = XtX_inverse.dot(XtY)
print(HatBeta)
```

Python

```
## [-1.96537261e+03  1.14933993e+00]
```

```julia
using LinearAlgebra
X = [ones(length(Year)) Year]
```

Julia

```
## 7×2 Matrix{Float64}:
##  1.0  1936.0
##  1.0  1946.0
##  1.0  1951.0
##  1.0  1963.0
##  1.0  1975.0
##  1.0  1997.0
##  1.0  2006.0
```

```
Y = CaloriesPerServingMean
```

```
## 7-element Vector{Float64}:
##  268.1
##  271.1
##  280.9
##  294.7
##  285.6
##  288.6
##  384.4
```

```
HatBeta = inv(X' * X) * X' * Y
```

```
## 2-element Vector{Float64}:
##  -1965.3726072602763
##      1.14933993399314
```

```
HatY = X * HatBeta
```

```
## 7-element Vector{Float64}:
##  259.74950495044277
##  271.2429042903741
##  276.98960396034
##  290.7816831682576
##  304.57376237617535
##  329.85924092402445
##  340.2033003299627
```

```
println(HatBeta)
```

```
## [-1965.3726072602763, 1.14933993399314]
```

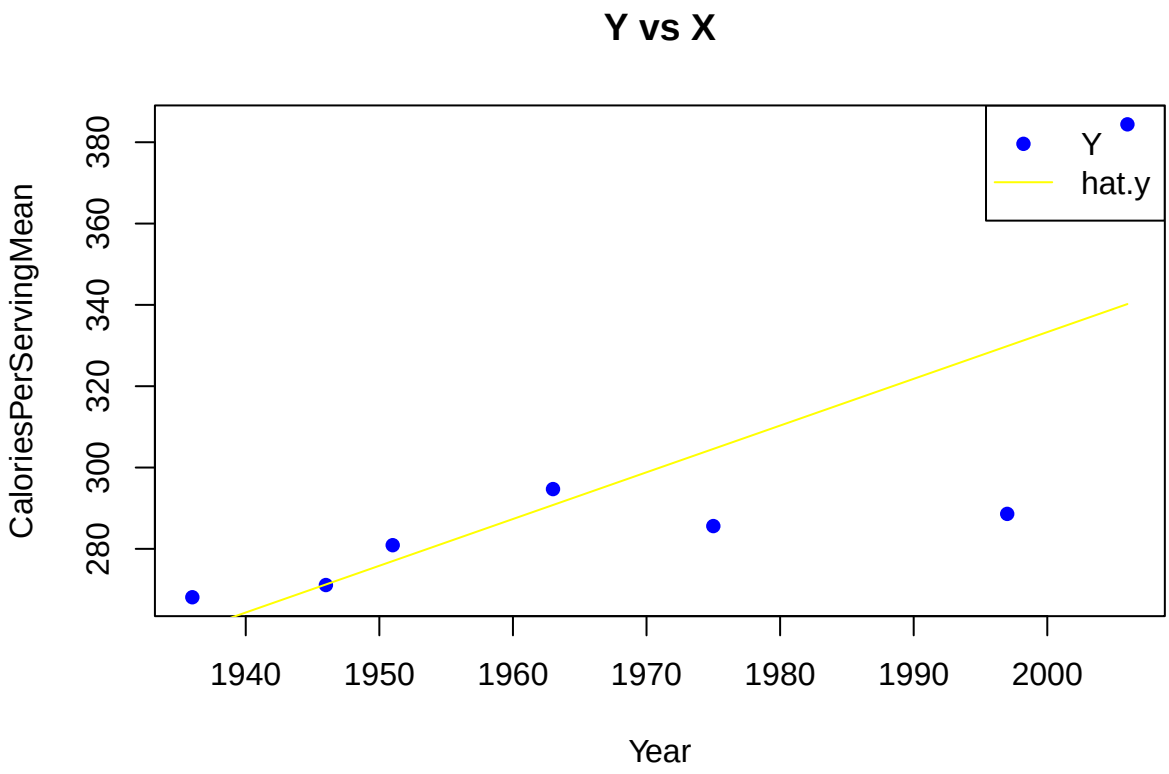
To check your work, calculate the values predicted by your statistical model. Compute `HatY` by multiplying `X` and `HatBeta`,

$$\widehat{y} = \mathbf{X}\widehat{\beta}$$

Plot `Y` vs the independent variable (the second column of `X`) as points, and `HatY` vs `X` as the independent variable as a line, preferably a different colors. The `HatY` values should fall in a straight line that interpolates `Y` values.

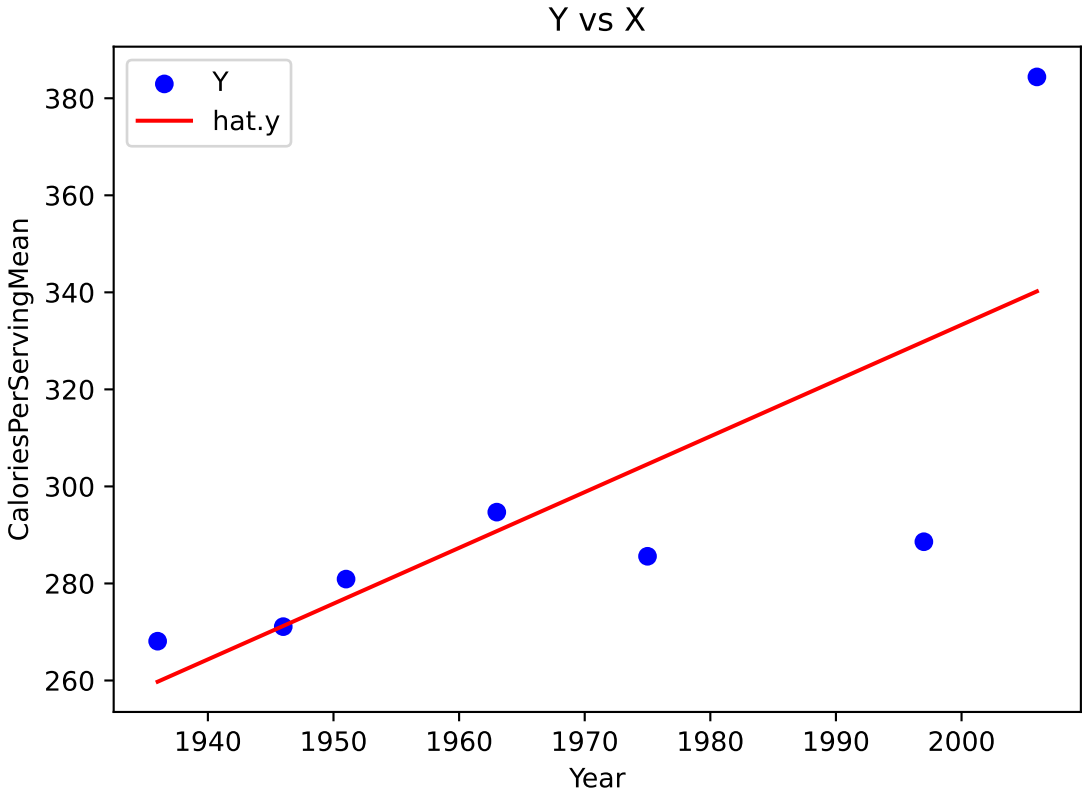
I had to call `.transpose().tolist()` to make `numpy` arrays conform to the arguments for `matplotlib`, but you might find a better solution for Python plots.

```
hat.y <- X%*%HatBeta
plot(x, Y, pch = 16, col = "blue", xlab = "Year", ylab = "CaloriesPerServingMean", main = "Y vs X")
lines(x, hat.y, col = "yellow")
legend("topright", legend = c("Y", "hat.y"), col = c("blue", "yellow"), pch = c(16, NA), lty = c(NA, 1))
```



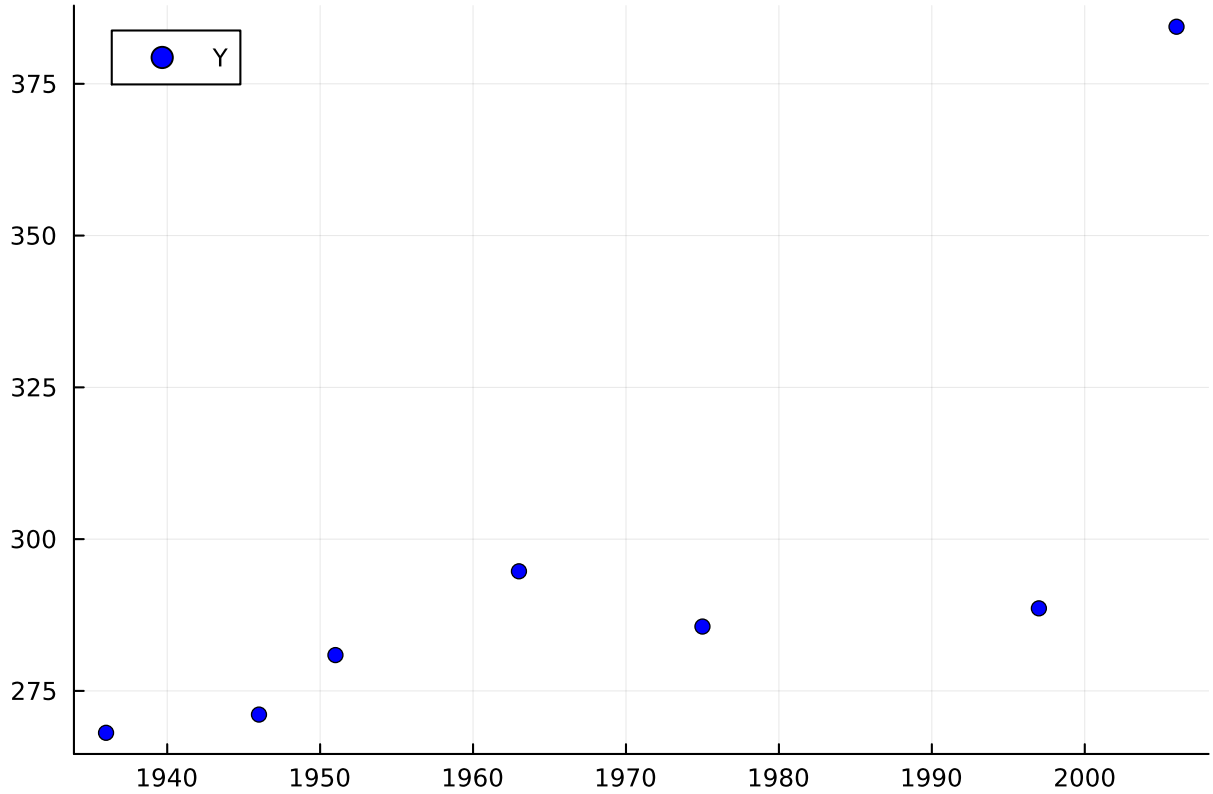
R

```
import numpy as np
import matplotlib.pyplot as plt
# Calculate HatY
hat_y = np.matmul(X, HatBeta)
# Plot Y vs X
plt.scatter(X[:, 1], Y, color='blue', marker='o', label='Y')
# Plot HatY vs X
plt.plot(X[:, 1], hat_y, color='red', label='hat.y')
# Set plot labels and title
plt.xlabel('Year')
plt.ylabel('CaloriesPerServingMean')
plt.title('Y vs X')
plt.legend()
plt.show()
```



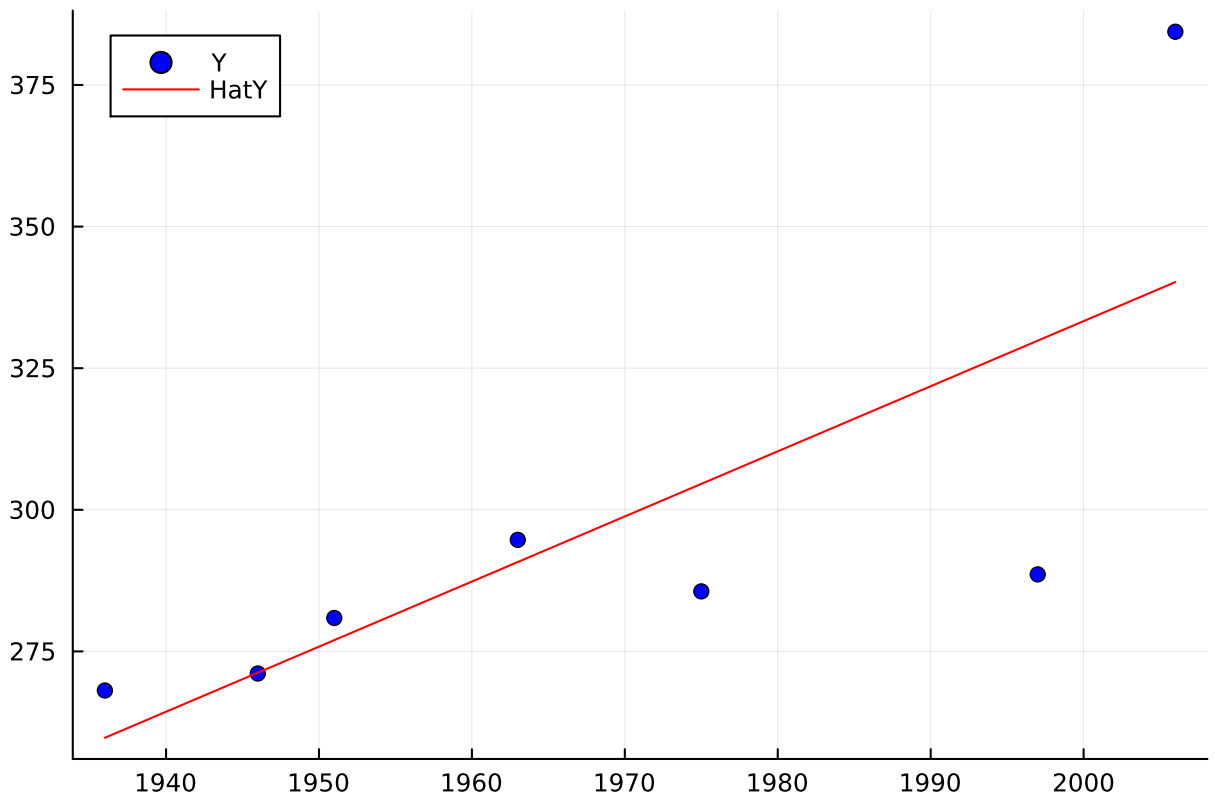
Python

```
using Plots
scatter(Year, Y, color = "blue", label = "Y")
```

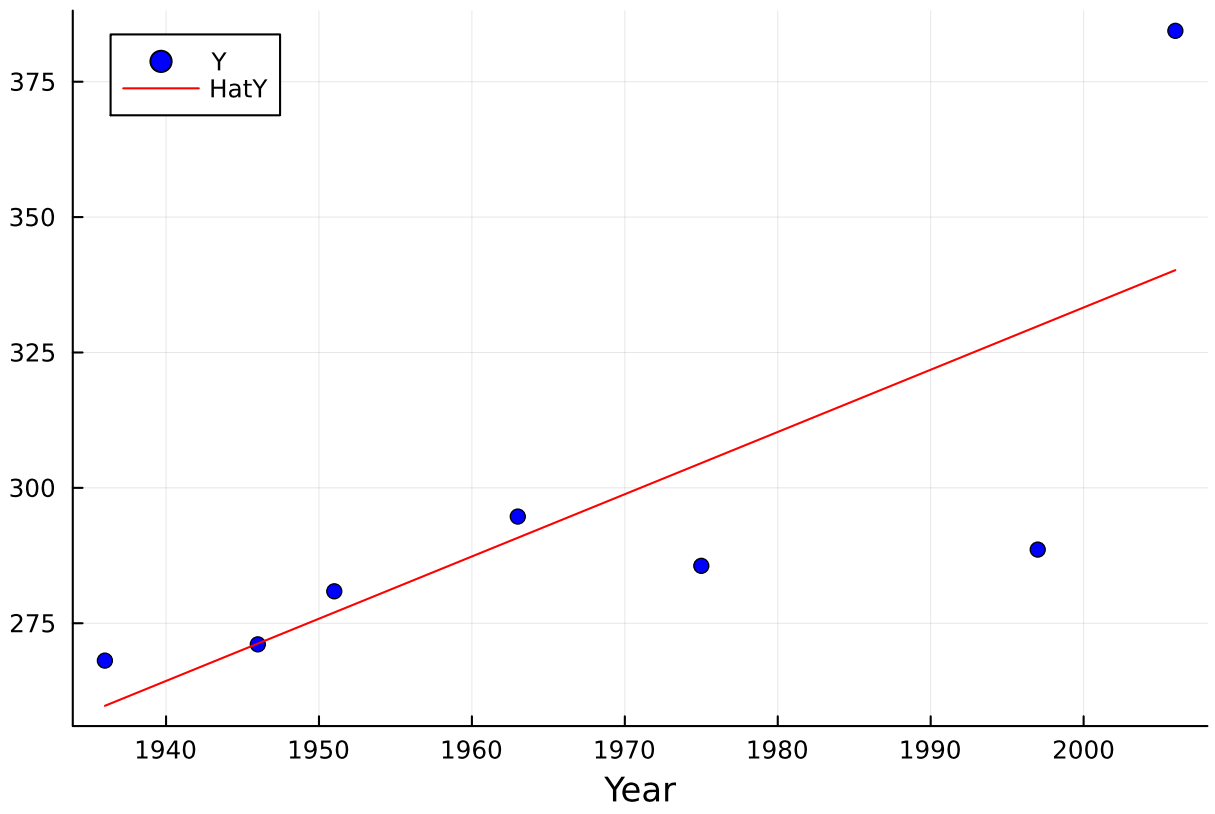


Julia

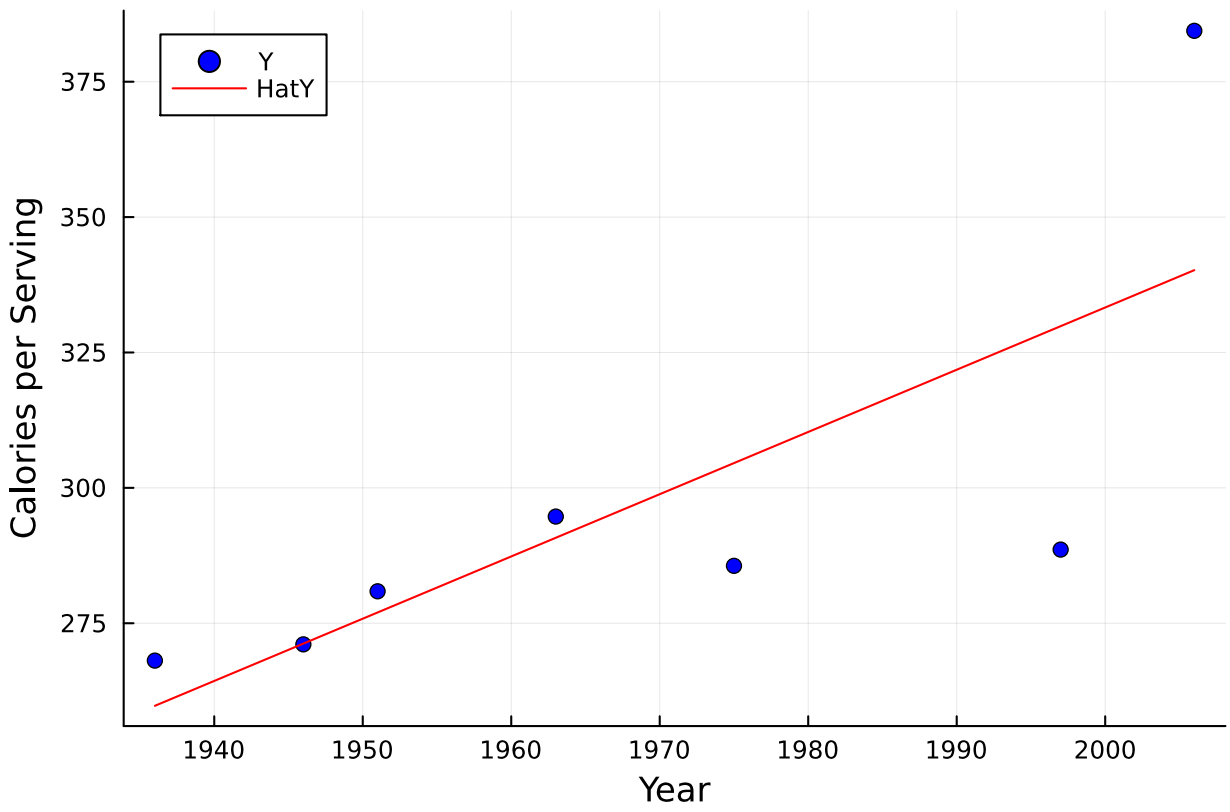
```
plot!(Year, HatY, color = "red", label = "HatY")
```




```
xlabel!("Year")
```




```
ylabel!("Calories per Serving")
```



You can also compare your result to the R, Python or Julia functions (set `eval=TRUE`).

```
summary(lm(CaloriesPerServingMean~Year))
```

```
##
## Call:
## lm(formula = CaloriesPerServingMean ~ Year)
##
## Residuals:
##       1       2       3       4       5       6       7
##  8.3505 -0.1429  3.9104  3.9183 -18.9738 -41.2592  44.1967
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1965.3726   875.8786  -2.244   0.0748 .
## Year            1.1493     0.4451   2.582   0.0493 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 28.69 on 5 degrees of freedom
## Multiple R-squared:  0.5715, Adjusted R-squared:  0.4858
## F-statistic: 6.668 on 1 and 5 DF,  p-value: 0.0493
```

```
import statsmodels.api
model = statsmodels.api.OLS(CaloriesPerServingMean, statsmodels.api.add_constant(Year))
results = model.fit()
print(results.summary())
```

```
using GLM
using DataFrames
data = DataFrame(CaloriesPerServingMean=CaloriesPerServingMean,Year=Year);
model = fit(LinearModel,
            @formula(CaloriesPerServingMean ~  Year),
            data)
```